

5/5/2026

Predicting Video Game Success (Hits)

Machine Learning Project

Instructor Name / Omar Alghushairy

Student Name	Student ID
Murad Alasmar	2140721
Bandar Farhan	2241471
Nawaf Aloufi	2240160
Faisal Alotaibi	2244074
Mohammed Baageel	2140067

Predicting Video Game Success (Hits)

1. Description of the Investigation	3
Objective:	3
Hypothesis:	3
Definition of a “Hit”:	3
Dataset Selection and Justification:	3
2. Methodology & Code Workflow	4
A. Data Preprocessing & Feature Engineering	4
1. Raw Data:	4
2. Missing Values:	4
3. Target Creation:	4
4. Data Leakage Prevention:	4
5. Encoding:	4
6. Splitting & Scaling:	4
B. Machine Learning Models	4
1. Models Used:	4
2. Class Imbalance:	4
3. Evaluation:	4
4. SVM Comparison:	4
C. Missing Values Analysis:	5
D. Outlier Detection (IQR Method):	5
Results and Model Evaluation	6
Logistic Regression:	6
Random Forest Classifier	7
Support Vector Machine (SVM) - Kernel Comparison:	8
Linear Kernel:	8
RBF Kernel:	9
Polynomial Kernel:	10

3. Full Model Results Comparison:	11
Final Model Selection	12
Best model for detecting Hit games	12
Best balanced model overall	12
Final Conclusion	12
4. Ethical Considerations	13
1. Stifling Innovation:	13
2. Bias Against Small Publishers:	13
3. Data Bias:	13
4. Over-Reliance on Automated Decisions:	13

Predicting Video Game Success (Hits)

1. Description of the Investigation

Objective:

The main objective of this project is to predict whether a video game will become a commercial “Hit” using only pre-release characteristics such as Platform, Genre, Publisher, and Release Year. This prediction helps simulate real-world decision-making before a game is launched, where publishers must estimate a game’s potential success without access to actual sales data.

Hypothesis:

It is hypothesized that certain features such as platform, genre, and publisher significantly influence whether a video game becomes a commercial hit. In particular, games released on popular platforms, belonging to high-demand genres, and published by well-established companies are more likely to achieve hit status.

Definition of a “Hit”:

For this investigation, a video game is classified as a Hit if it achieves 1.0 million or more in Global Sales. This threshold transforms the original continuous sales variable into a binary target variable called `IsItHit`, allowing the problem to be approached as a supervised classification task.

Dataset Selection and Justification:

This project uses the Video Game Sales dataset (`vgsales.csv`), which contains historical sales records of video games across multiple platforms, genres, publishers, and release years. The dataset was selected because it includes both descriptive pre-release features and post-release sales outcomes, making it suitable for building a predictive model for commercial success. It was obtained from publicly available machine learning repositories, such as Kaggle, and was chosen due to its large sample size, structured format, and strong relevance to the project objective.

2. Methodology & Code Workflow

The project was implemented using Python with libraries such as Pandas, NumPy, and Matplotlib.

A. Data Preprocessing & Feature Engineering

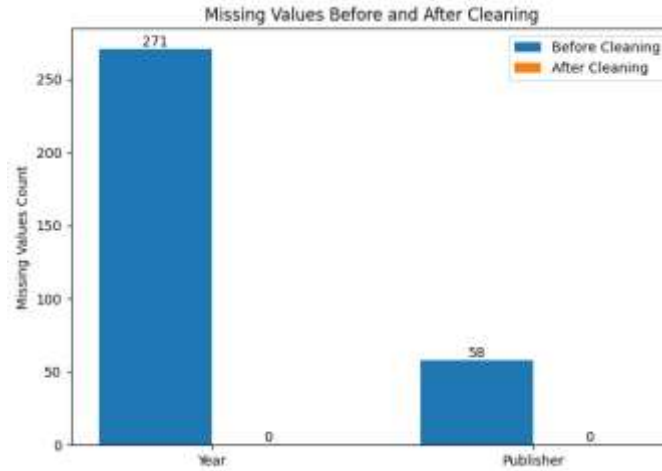
1. Raw Data:
The vgsales.csv dataset was loaded for analysis.
2. Missing Values:
Rows with missing Year or Publisher values were removed.
3. Target Creation:
A binary variable IsItHit was created where $\text{Global_Sales} \geq 1.0$.
4. Data Leakage Prevention:
All sales-related columns, along with Rank and Name, were removed to avoid leakage.
5. Encoding:
Categorical features were converted using One-Hot Encoding.
6. Splitting & Scaling:
The data was split into 70% training and 30% testing sets, and features were scaled using StandardScaler.

B. Machine Learning Models

1. Models Used:
Logistic Regression, Random Forest, and SVM were implemented.
2. Class Imbalance:
`class_weight='balanced'` was applied to handle imbalance.
3. Evaluation:
Models were evaluated using accuracy and confusion matrices on the test set.
4. SVM Comparison:
Different kernels (Linear, RBF, Polynomial) were tested.

C. Missing Values Analysis:

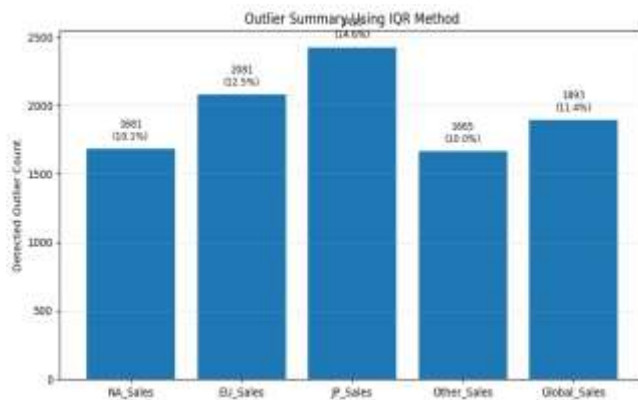
Before cleaning, the dataset contained missing values in the Year and Publisher columns (271 and 58 respectively). After removing these records, no missing values remained, ensuring a clean dataset for model training.



D. Outlier Detection (IQR Method):

Outliers were identified using the Interquartile Range (IQR) method across sales-related features. As shown in the outlier summary figure, a significant number of outliers exist in all regions, especially in JP_Sales and EU_Sales.

However, these outliers were not removed, as they represent real-world high-selling games, which are important for predicting “Hit” outcomes.



Outlier Summary Using IQR Method						
① Outliers were detected only; no rows were removed.						
Column	Q1	Q3	IQR	Lower Bound	Upper Bound	Outlier Count
NA_Sales	0.00	0.24	0.24	-0.360	0.600	1681
EU_Sales	0.00	0.11	0.11	-0.165	0.275	2081
JP_Sales	0.00	0.04	0.04	-0.060	0.100	2425
Other_Sales	0.00	0.04	0.04	-0.060	0.100	1665
Global_Sales	0.06	0.47	0.41	-0.555	1.085	1893

IQR = Q3 - Q1

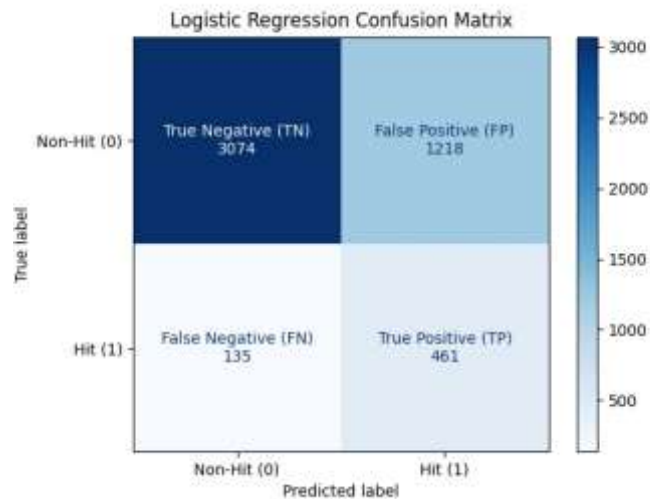
Results and Model Evaluation

Logistic Regression:

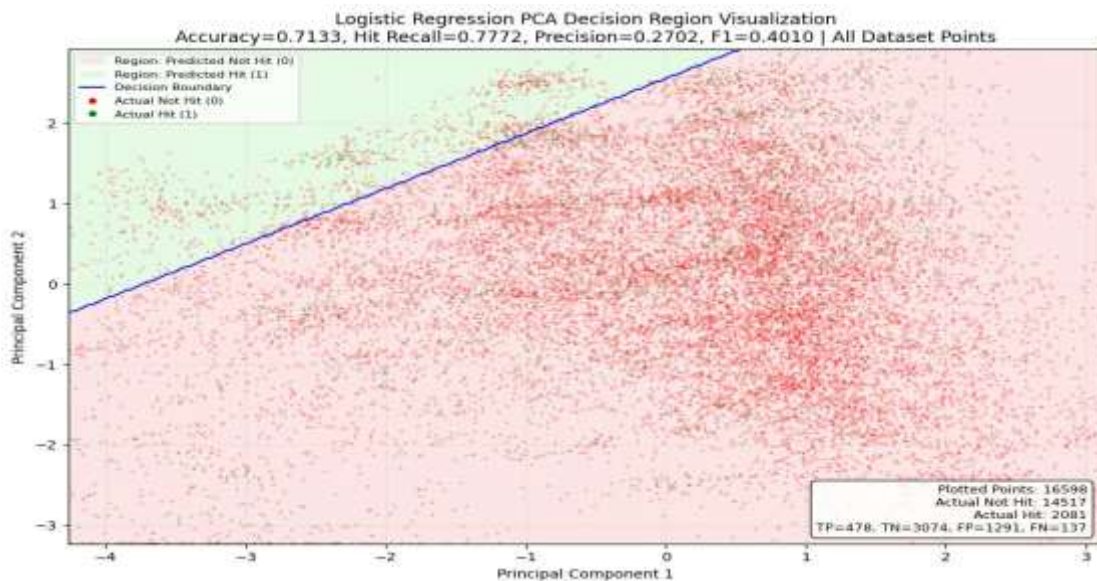
Accuracy: 71.33%

Result:

`[[3074, 1218],`
`[135, 461]]`



Analysis: Provided a decent baseline. It correctly identified 461 hits but suffered from a high number of false positives (1218 games incorrectly predicted as hits).



This visualization shows a linear decision boundary separating hit and non-hit games. While the model performs reasonably well, there is noticeable overlap between classes, leading to a significant number of misclassifications.

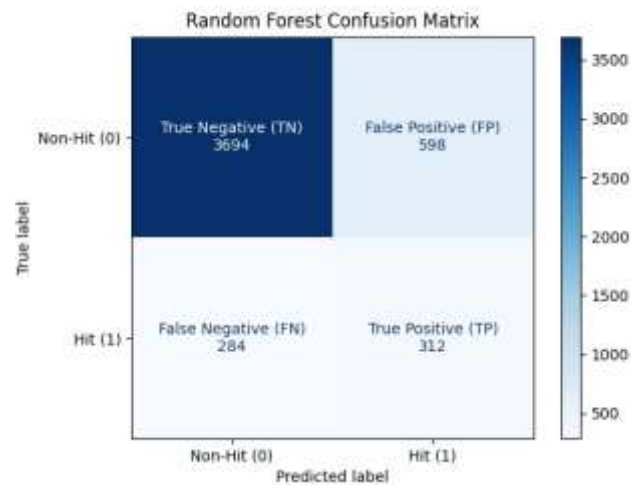
Random Forest Classifier (N=50,threshold=0.25):

Accuracy: 81.08%

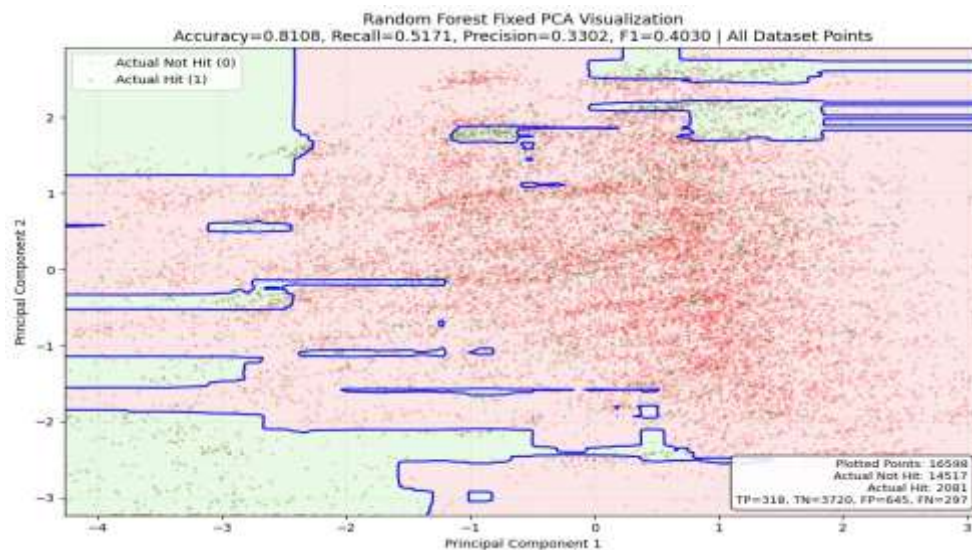
Result:

`[[3694, 598],`

`[284, 312]]`



Analysis: Achieved the highest overall accuracy. By adjusting the hit threshold, it balanced the detection of true hits while significantly minimizing false positives compared to Logistic Regression.



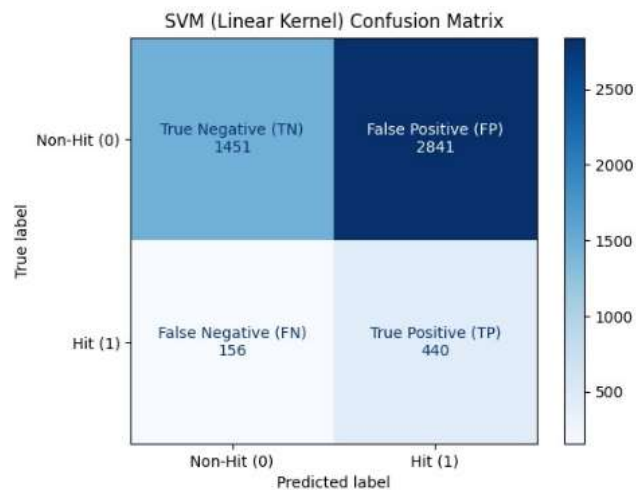
The Random Forest model produces more flexible and complex decision boundaries, capturing non-linear patterns in the data. This allows for better classification performance compared to simpler models.

Support Vector Machine (SVM) - Kernel Comparison:

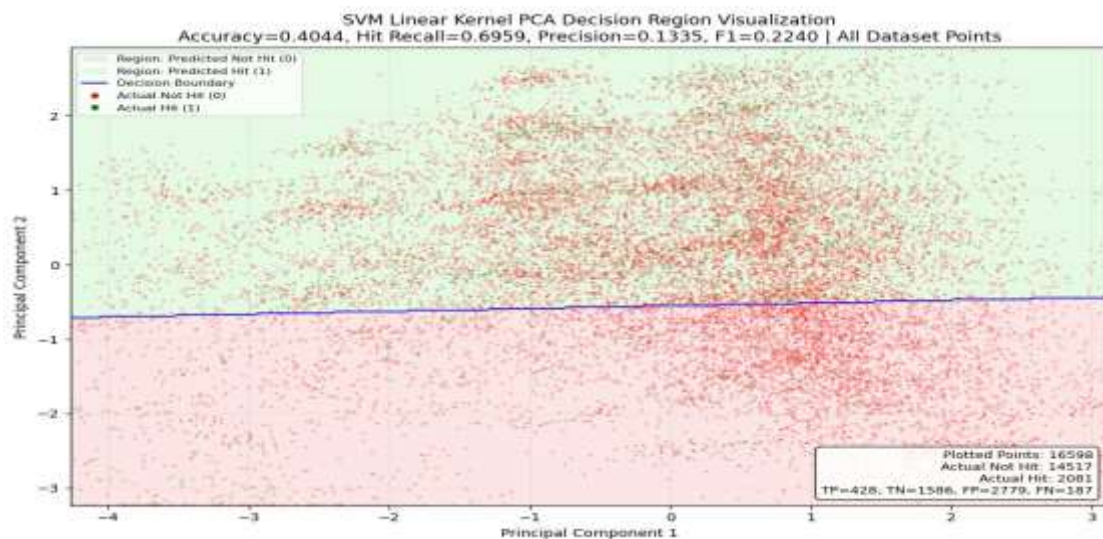
Linear Kernel:

40.44% Accuracy

`[[1451, 2841],`
`[156, 440]]`



The linear SVM shows moderate performance, correctly identifying some hits and non-hits. However, the high number of false positives indicates that the model struggles to accurately distinguish between the two classes.

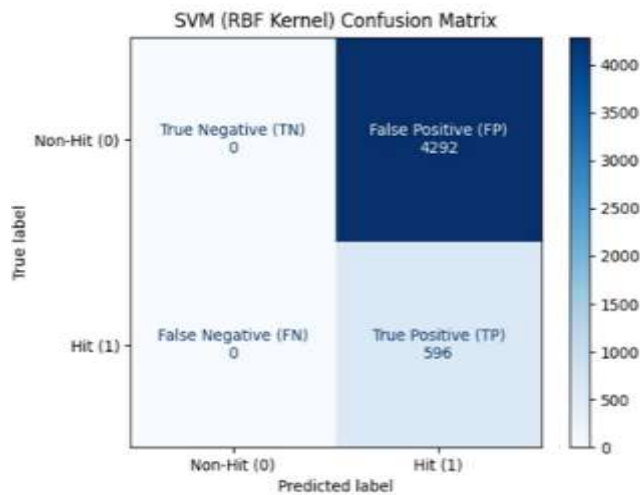


The linear SVM creates a simple straight decision boundary, which is insufficient to separate the complex distribution of the data, resulting in moderate performance.

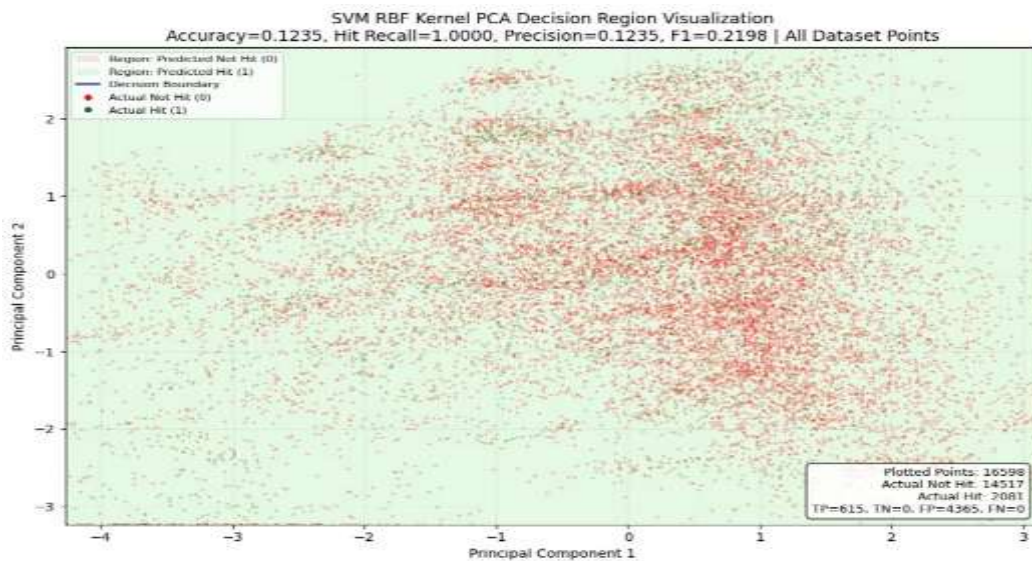
RBF Kernel:

12.35% Accuracy

`[[0, 4292],`
`[0, 596]]`



The RBF kernel performs extremely poorly, predicting nearly all samples as “Hit.” This results in zero true negatives and a very high number of false positives, making the model unreliable.

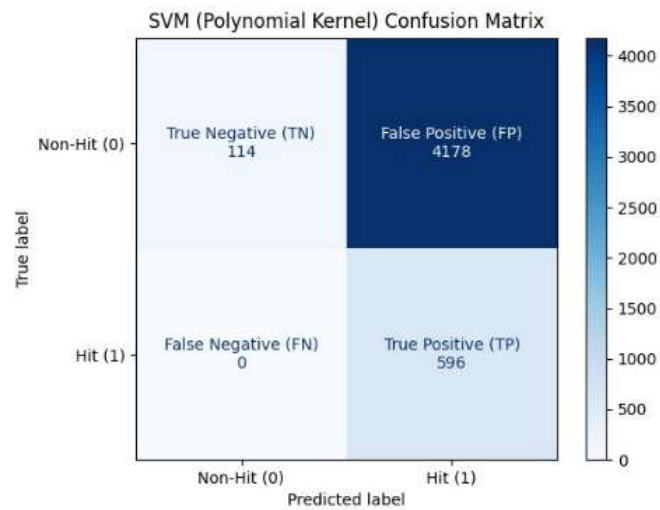


The RBF kernel attempts to model non-linear relationships, but in this case it over-predicts one class, leading to extremely poor accuracy and unreliable classification results.

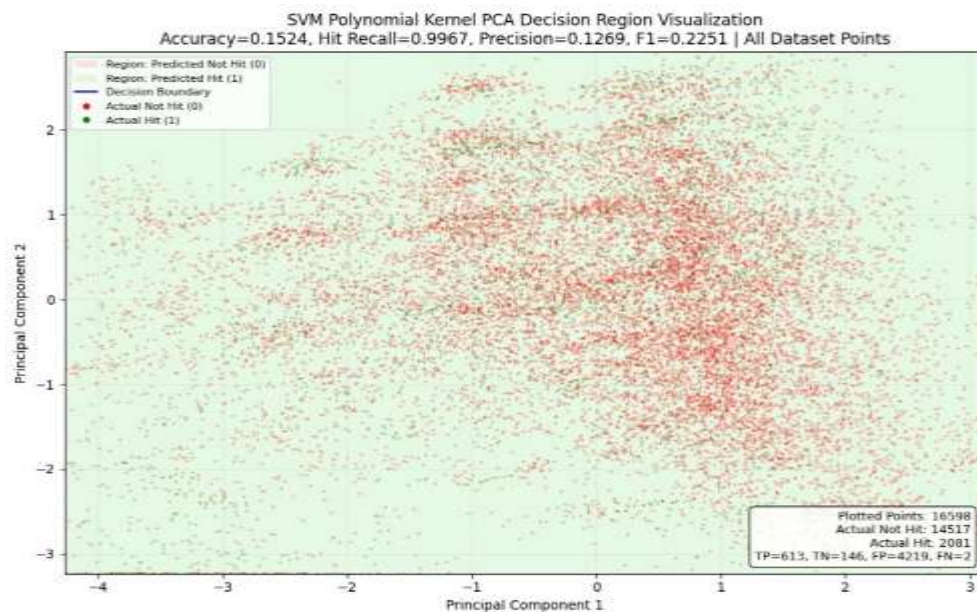
Polynomial Kernel:

15.24% Accuracy

`[[114, 4178],
[0, 596]]`



The polynomial kernel also performs poorly, with very few true negatives and a large number of false positives. Although it correctly identifies most hits, it fails to properly classify non-hit games.



The polynomial kernel produces more complex decision boundaries; however, it fails to generalize effectively, resulting in high misclassification and low overall performance.

3. Full Model Results Comparison:

To address the inherent class imbalance in the dataset, we tested multiple sampling approaches. The table below compares the performance of the five machine learning models across all five distinct data handling cases (Original, Undersampling, and Oversampling). Because relying solely on accuracy can be misleading when dealing with imbalanced data, we evaluated the models using the following key metrics:

Case	Model	Accuracy	Hit Recall	Hit Precision	Hit F1-score
Original / Normal	Logistic Regression	0.7133	0.7772	0.2702	0.4010
Original / Normal	Random Forest	0.8108	0.5171	0.3302	0.4030
Original / Normal	SVM Linear	0.4044	0.6959	0.1335	0.2240
Original / Normal	SVM RBF	0.1235	1.0000	0.1235	0.2198
Original / Normal	SVM Polynomial	0.1524	0.9967	0.1269	0.2251
Undersampling 8000	Logistic Regression	0.7112	0.7756	0.2684	0.3988
Undersampling 8000	Random Forest	0.7940	0.5626	0.3137	0.4028
Undersampling 8000	SVM Linear	0.5359	0.6748	0.1643	0.2642
Undersampling 8000	SVM RBF	0.1235	1.0000	0.1235	0.2198
Undersampling 8000	SVM Polynomial	0.1737	0.9919	0.1292	0.2287
Undersampling 6000	Logistic Regression	0.7102	0.7789	0.2682	0.3990
Undersampling 6000	Random Forest	0.7643	0.6423	0.2928	0.4022
Undersampling 6000	SVM Linear	0.4659	0.7301	0.1526	0.2524
Undersampling 6000	SVM RBF	0.1235	1.0000	0.1235	0.2198
Undersampling 6000	SVM Polynomial	0.2014	0.9886	0.1328	0.2342
Oversampling 4000	Logistic Regression	0.7080	0.7691	0.2650	0.3942
Oversampling 4000	Random Forest	0.7863	0.5870	0.3083	0.4043
Oversampling 4000	SVM Linear	0.4357	0.7366	0.1461	0.2438
Oversampling 4000	SVM RBF	0.1235	1.0000	0.1235	0.2198
Oversampling 4000	SVM Polynomial	0.1717	0.9919	0.1290	0.2283
Oversampling 8000	Logistic Regression	0.7064	0.7642	0.2630	0.3913
Oversampling 8000	Random Forest	0.7773	0.6016	0.2998	0.4002
Oversampling 8000	SVM Linear	0.3225	0.8650	0.1392	0.2397
Oversampling 8000	SVM RBF	0.1235	1.0000	0.1235	0.2198
Oversampling 8000	SVM Polynomial	0.1773	0.9902	0.1296	0.2292

Final Model Selection

Best model for detecting Hit games

The best model for detecting Hit games is Logistic Regression because it achieved the most stable and useful Hit Recall across all experiments, with Hit Recall around 76% to 78% while keeping accuracy around 70% to 71%. This makes it suitable when the goal is to avoid missing successful games.

Best balanced model overall

The best balanced model is Random Forest with Oversampling Hit to 4000 because it achieved the best Random Forest F1-score of 0.4043 while keeping accuracy reasonably high at 0.7863. It also improved Hit Recall compared to the original Random Forest from 0.5171 to 0.5870.

Final Conclusion

The results show that accuracy alone is not enough for this dataset because the data is imbalanced. The original Random Forest achieved the highest accuracy, but its Hit Recall was relatively low, meaning it missed many successful games. Logistic Regression achieved the best and most stable Hit Recall, making it the best choice for detecting Hit games. Random Forest improved after applying sampling techniques, especially with Oversampling Hit to 4000, which gave the best balance between accuracy and F1-score. On the other hand, SVM RBF and SVM Polynomial produced misleading recall scores because they classified almost all samples as Hit, resulting in very low accuracy and precision.

Therefore, the recommended final model depends on the project goal. If the priority is detecting as many Hit games as possible, Logistic Regression is the best option. If the priority is a more balanced model with stronger accuracy and F1-score, Random Forest with Oversampling Hit to 4000 is the best choice.

4. Ethical Considerations

If this Machine Learning model were used by real game developers to support decision-making and funding allocation, several ethical and professional concerns must be considered:

1. Stifling Innovation:

If the model consistently favors certain genres or platforms (such as “Action” games on “PlayStation”), developers may prioritize only those categories. This could discourage experimentation and creativity, limiting the development of innovative or unique games. Over time, this may lead to a repetitive gaming industry with reduced diversity in content.

2. Bias Against Small Publishers:

Since the model is trained on historical data, it tends to favor large, well-established publishers that have previously achieved commercial success. Smaller or independent developers, who may not have a strong historical track record, could be unfairly classified as producing “Non-Hit” games. This may reduce their chances of receiving funding, reinforcing existing inequalities within the industry.

3. Data Bias:

The dataset used in this project is based on past sales data, which may not accurately reflect current market trends or player preferences. As the gaming industry evolves rapidly, relying on outdated data can lead to inaccurate predictions. This highlights the importance of continuously updating datasets to ensure model relevance and fairness.

4. Over-Reliance on Automated Decisions:

Relying too heavily on machine learning predictions may reduce human judgment in decision-making processes. Game development involves creativity, innovation, and market intuition, which cannot be fully captured by data alone. Therefore, such models should be used as supportive tools rather than definitive decision-makers.